Proyecto Final Informática Gráfica

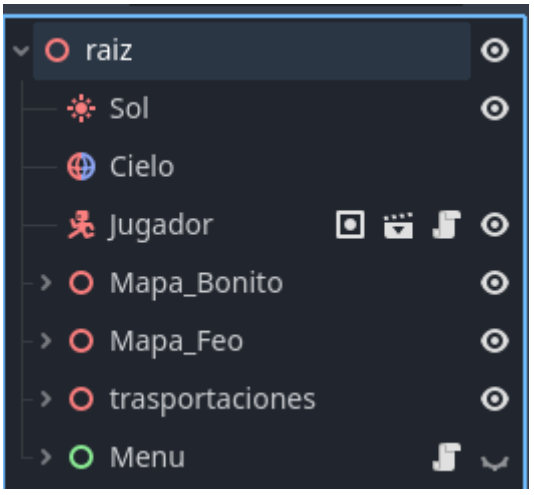
Diana Peven Kobtseva - 3A (Grupo 2)

Descripción

El juego que he creado se trata de de un juego donde controlas un personaje en primera persona, al principio apareces en un paisaje bonito, como un jardín, pero en ese paisaje hay una bola de luz que llama la atención, la cual está hecha para atraer al personaje, una vez te acercas y tomas contacto con la bola, te teletransportas a un mundo totalmente distinto, una especie de infierno, apareces en una plataforma y entonces debes recorrer un camino por plataformas para llegar a tu destino, donde algunas se mueven y otras simplemente están estáticas, una vez recorres este camino llegas a una especie de arena donde hay un gran ser demoníaco, al que primero debes matar con un cuchillo que cogerás al llegar al destino, y después de morir este ser hará que aparezca un bote de pastillas, si te acercas al bote de pastillas para cogerlas, cuando las toques te volverás a transportar al jardín del inicio, y allí el juego habrá terminado. Puedes salir en cualquier momento pulsando la tecla ESC, donde se abrirá un menú en el que puedes elegir continuar o salir.

Estructura principal

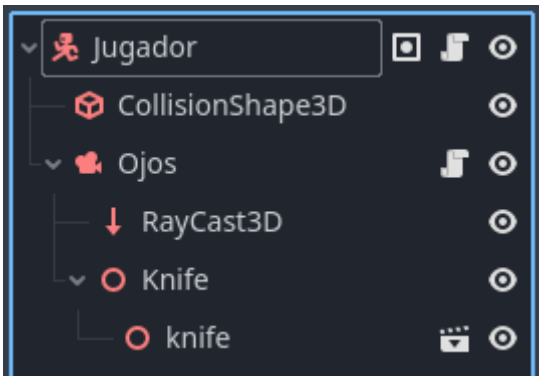
La estructura principal del juego es la siguiente:

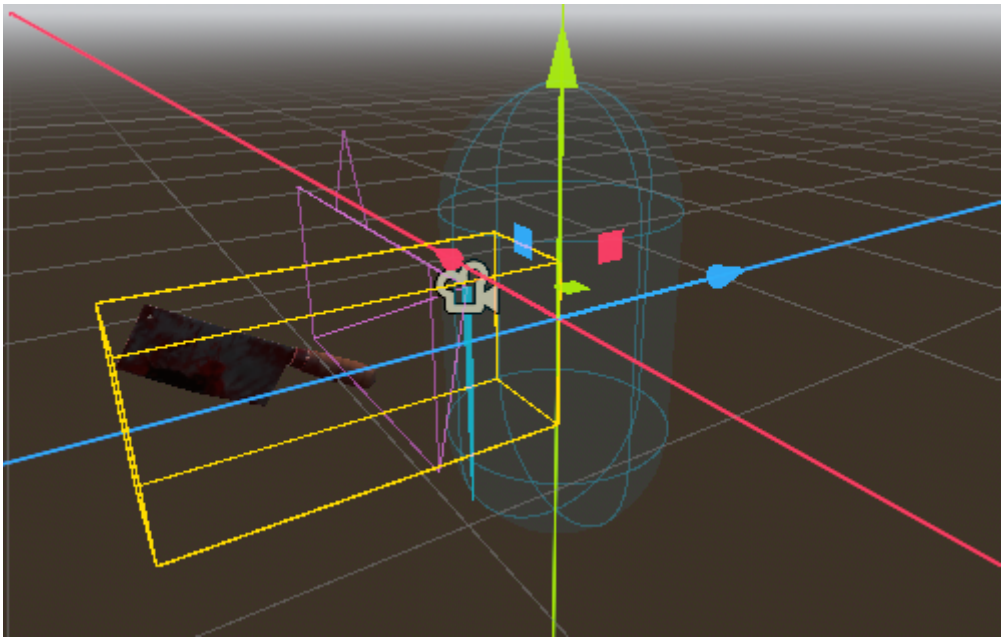


Donde primeramente tenemos un sol y un cielo, que servirán para dar ambientación y luz a los mapas, tenemos al jugador, que es el protagonista del juego, el nodo del mapa bonito, que es donde aparecemos al principio y al final del juego, el nodo del mapa feo, que es donde se desarrolla la mayor parte del juego, las transportaciones, donde guardo las áreas y posiciones para trasladar al personaje de mapa en mapa,

y el menú, que será la interfaz para continuar o salir del juego.

1. Jugador





El jugador es un CharacterBody3D, este tiene asignado un CollisionShape3D para poder moverse y colisionar con el suelo, las barreras, etc.
Tiene asignada una Camera3D que son los ojos del personaje, a su vez los ojos tienen un RayCast3D, que nos servirá para simular los ataques del jugador.
También tenemos un cuchillo en nuestros ojos para que se mueva junto con nosotros, pero este solo será visible y usable cuando lo cojamos en el mapa.

1.1 Script del jugador

- Botones



Tenemos estos botones para controlar el movimiento del personaje, nos moveremos con WSAD, saltaremos con SPACE y correremos con SHIFT.

- Declaración de variables

```
extends CharacterBody3D

@export var velocidad: float = 5.0
@export var velocidad_correr: float = 9.0
@export var salto: float = 7.0
@export var sensibilidadRaton: float = 0.001

@onready var ojos: Camera3D = $Ojos
@onready var capturado := true

@export var saltos_maximos := 2
var saltos_actuales := 0

var pitch: float = 0.0 #tope de camara hacia arriba

var has_knife := false
@onready var knife: Node3D = $Ojos/Knife

@onready var menu: Control = $"../Menu"
```

Estas variables nos servirán para las funciones, tenemos variables para la velocidad de movimiento, saltos, la sensibilidad del ratón y si este está capturado o no, tope de la cámara, los ojos del jugador, el cuchillo y un booleano para el uso del cuchillo y el menú.

- Función _ready()

```
func _ready() -> void:
    Input.mouse_mode = Input.MOUSE_MODE_CAPTURED
    knife.visible = false
```

Ponemos el ratón a modo capturado por defecto y el cuchillo invisible.

- Función _input()

```
func _input(event) -> void:
    if event is InputEventMouseMotion and capturado:
        #aqui movemos la camara horizontalmete
        rotate_y(-event.relative.x * sensibilidadRaton)
        #aqui la movemos  verticalmente
        pitch = clamp(pitch - event.relative.y * sensibilidadRaton, deg_to_rad(-89), deg_to_rad(89))
        ojos.rotation.x = pitch

    #si apretamos ESC pasamos al modo captura
    if event is InputEventKey and event.pressed and event.keycode == KEY_ESCAPE:
        if capturado:
            capturado = false
            Input.mouse_mode = Input.MOUSE_MODE_VISIBLE
            menu.mostrar()
            capturado = true
```

Si el evento de entrada es movimiento del ratón y el ratón está capturado: giramos la cámara horizontalmente por el eje y con rotate_y, multiplicando el movimiento en x por la sensibilidad para controlar la velocidad.
Luego se ajusta pitch para la rotación vertical, multiplicando el movimiento del eje y por la sensibilidad del ratón, y usando clamp limitamos el ángulo de giro. Esa rotación vertical se aplica a ojos.rotation.x.

Si se pulsa ESC: Se libera el ratón, se hace visible y se muestra el menú… Y después de usar el menú se vuelve a marcar capturado.

- Función _physics_process(delta: float)

```gdscript
func _physics_process(delta: float) -> void:
    var direccion = Vector3.ZERO

    if Input.is_action_pressed("mover_adelante"):
        direccion -= transform.basis.z

    if Input.is_action_pressed("mover_atras"):
        direccion += transform.basis.z

    if Input.is_action_pressed("mover_derecha"):
        direccion += transform.basis.x

    if Input.is_action_pressed("mover_izquierda"):
        direccion -= transform.basis.x

    direccion = direccion.normalized() #para que no aumente la velocidad por pulsar wd a la vez
```

Creo una dirección como un vector vacío, y luego, según la tecla que se pulse nos movemos a una dirección o a otra usando la orientación del personaje.
Luego normalizamos la dirección para evitar que la velocidad se multiplique, si no que se mantenga estable.

```gdscript
    var velocidad_actual := velocidad

    if Input.is_action_pressed("correr") and is_on_floor():
        velocidad_actual = velocidad_correr

    velocity.x = direccion.x * velocidad_actual
    velocity.z = direccion.z * velocidad_actual

    if is_on_floor():
        saltos_actuales = saltos_maximos
    else:
        velocity.y -= 11 * delta

    if Input.is_action_just_pressed("saltar") and saltos_actuales > 0:
        velocity.y = salto
        saltos_actuales -= 1

    move_and_slide()
```

Establecemos la velocidad actual, que cambia si se pulsa la acción de correr (y siempre que estemos en el suelo) y le damos al jugador la velocidad en x y en multiplicando por la dirección.

Luego, si estamos en el suelo tendremos la capacidad de realizar 2 saltos más, y si no, saltaremos con una gravedad de 11 (le di este valor para que sea más jugable)
Si pulsamos la acción de saltar y todavía no hemos gastado los dos saltos, lo hacemos, y restamos un salto.
Y move_and_slide() permite hacer todo el movimiento del personaje.

- Función can_attack()

```
func can_attack() -> bool:
    return has_knife
```

Para saber si podremos usar o no el cuchillo

- Función pickup_knife()

```
81  func pickup_knife() -> void:
82      has_knife = true
83      knife.visible = true
```

Se llamará cuando nos acerquemos al cuchillo en el mapa, establece has_knife a true y hace visible el cuchillo.

- Función drop_knife()

```
func drop_knife() -> void:
    has_knife = false
    knife.visible = false
```

Se llamará cuando nos acerquemos a las pastillas/cambiemos de mapa, establece has_knife a false y hace invisible el cuchillo.

1.2 Script de los ojos

-Variables

```
1   extends Camera3D
2
3   @export var ray_length: float = 5.0
4   @onready var ray: RayCast3D = $RayCast3D
5
6   @onready var player := get_parent()
```

Tenemos la longitud del rayo, la cual es corta para solo pegar de cerca ya que el cuchillo es un arma corta, tenemos el rayo y la variable del jugador.

- Función _input(event: InputEvent)

```
func _input(event: InputEvent) -> void:
    if event is InputEventMouseButton \
    and event.pressed \
    and event.button_index == MOUSE_BUTTON_LEFT:

        var mouse_pos = get_viewport().get_mouse_position()

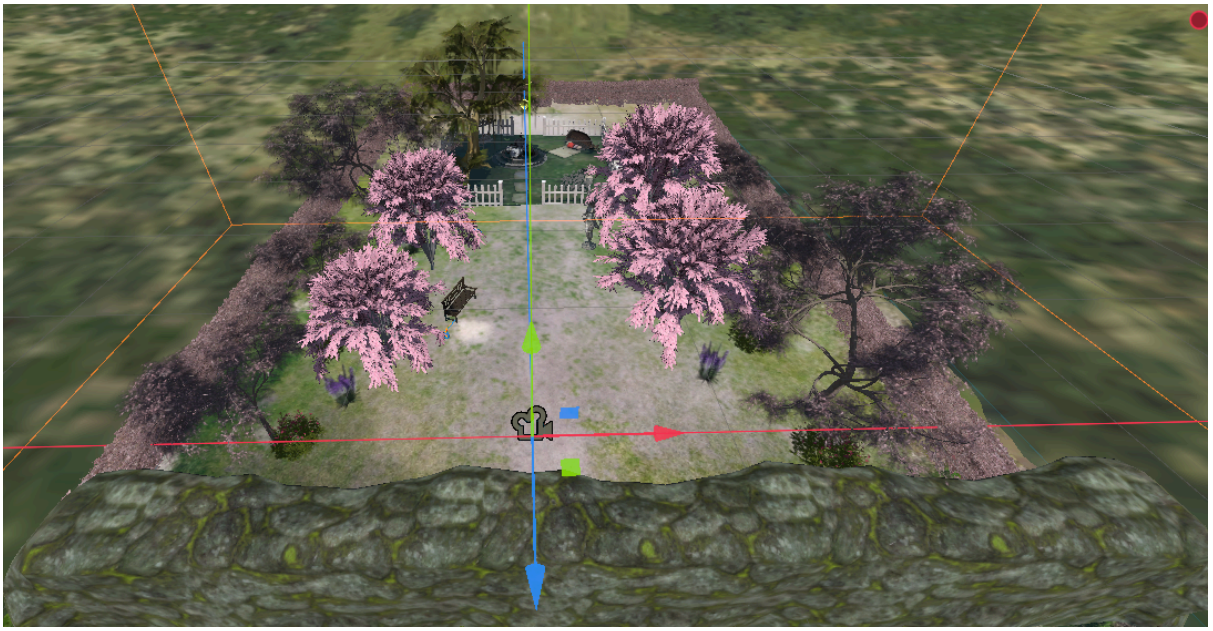
        ray.global_position = project_ray_origin(mouse_pos)
        ray.target_position = project_local_ray_normal(mouse_pos) * ray_length
        ray.force_raycast_update()

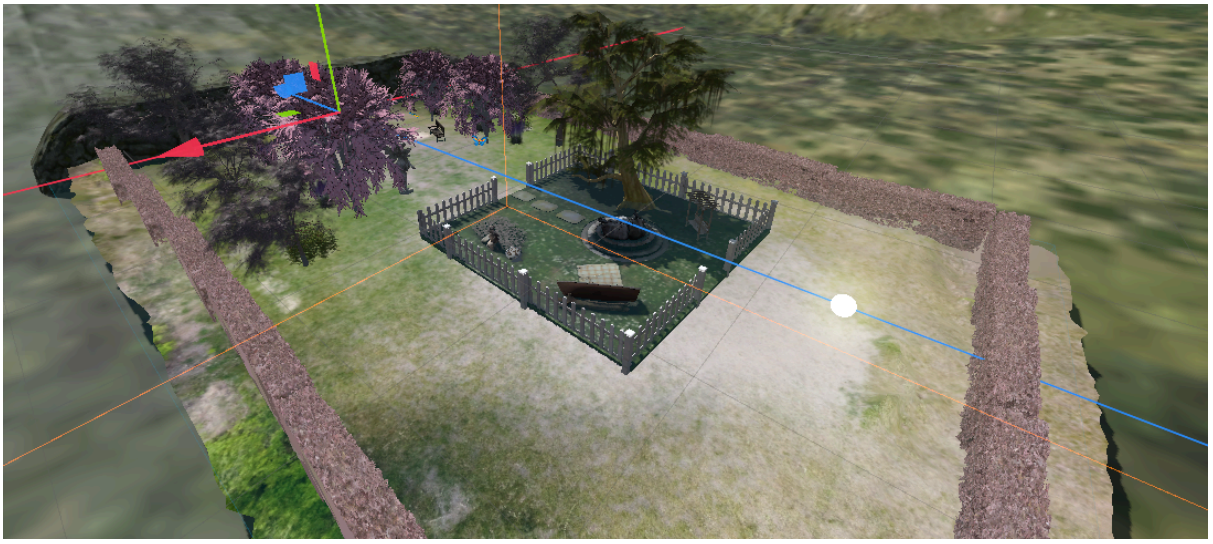
        if ray.is_colliding():
            var collider = ray.get_collider()

            # OPCIÓN PRO: grupos, no nombres
            if collider.is_in_group("enemy"):
                if not player.can_attack():
                    return
                collider.take_damage(25.0)
```

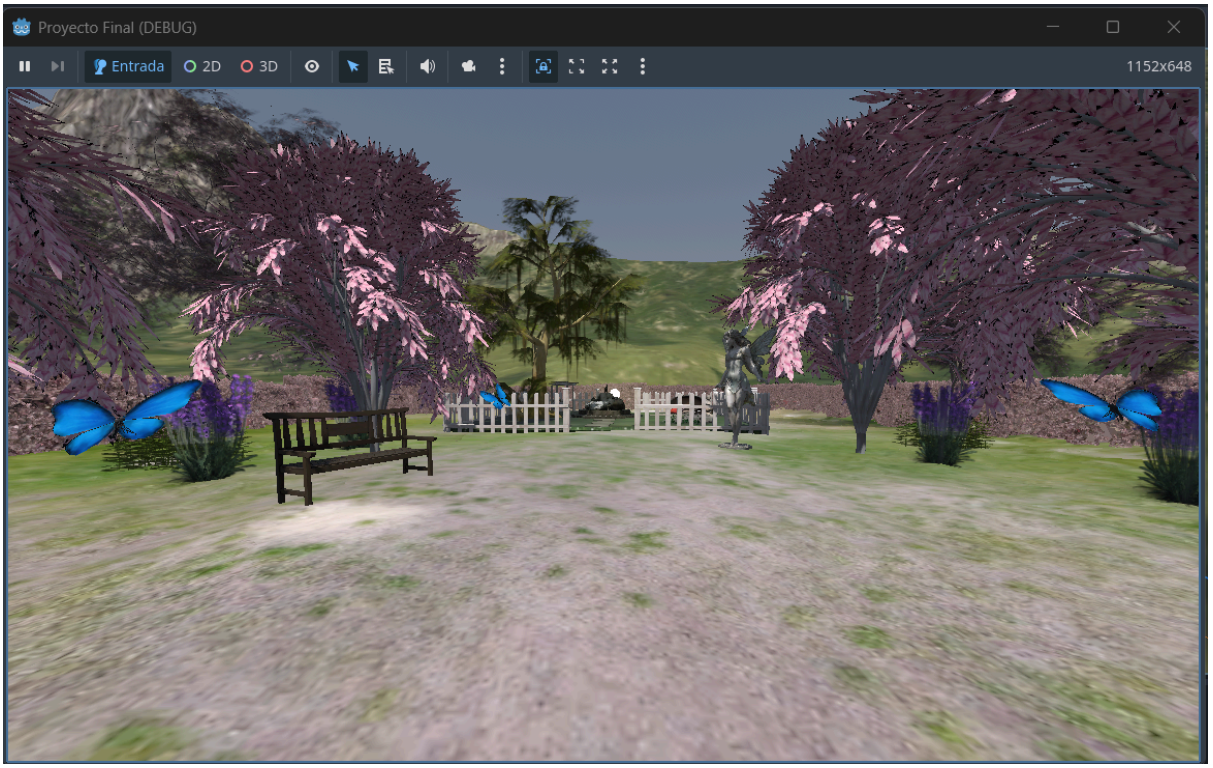
Si hacemos click izquierdo con el ratón proyectamos un rayo hacia la dirección del ratón, osea, hacia donde mira el jugador, y si choca, comprobamos que choque con el enemigo, el cual se encuentra en el grupo enemy (a su vez el jugador se encuentra en el grupo player) y si es afirmativo comprobamos si el jugador puede atacar (tiene el cuchillo) y si no puede no hacemos nada, si puede llamamos a la función take_damage del enemigo y le hacemos un daño de 25.

2. Mapa bonito

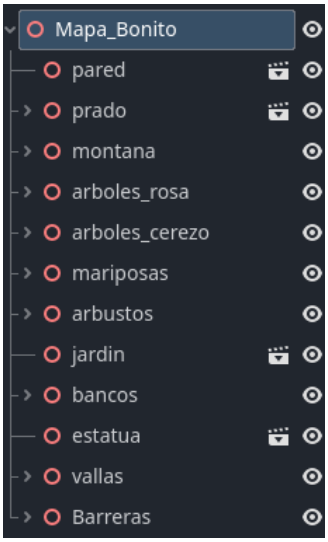




En partida:



Estructura general del mapa:

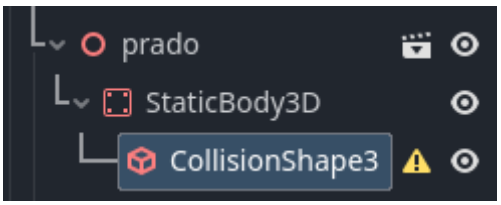


Estructura de cada nodo:



En general el mapa está constituido por muchos modelos importados, prácticamente todo lo es.

- El prado además tiene un CollisionShape el cual es el suelo:

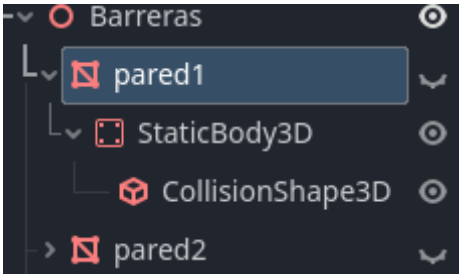


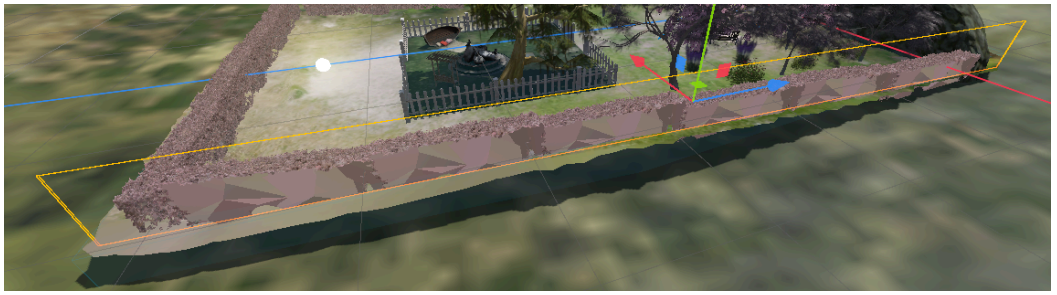


- Las mariposas tiene un script para permitir el movimiento de su animación:

```
1   extends Node3D
2
3   @onready var anim = $"AnimationPlayer"
4
5   func _ready():
6       anim.get_animation("Flying").loop = true
7       anim.play("Flying")
```

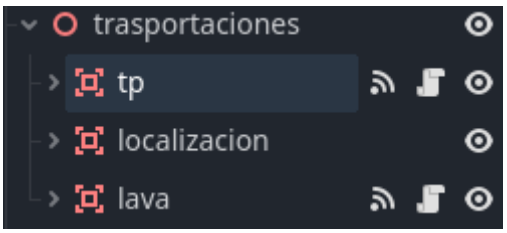
- Y tenemos unas barreras que limitan el mapa para no salirnos:





3. Teletransportación (1ª parte)

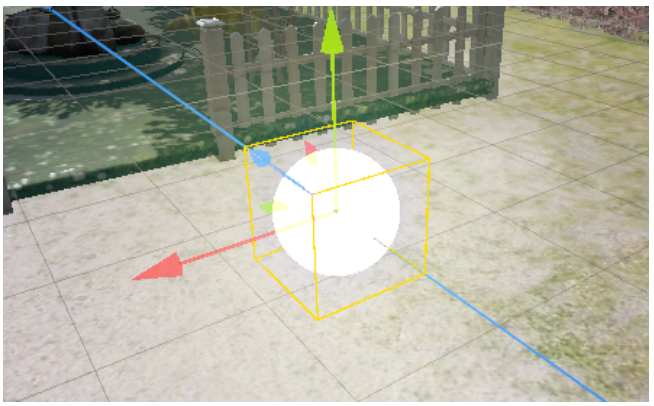
Antes de pasar al siguiente mapa, es importante hablar sobre la teletransportación entre ambos mapas, en la escena raíz tenemos este nodo:



Que nos permitirá cambiar de posición en algunos lugares. Más en concreto hablaremos del Area3D “tp”:



Cree un Area3d y la situé en el mapa bonito para transportarnos de este al otro, le asigné un CollisionShape3D y un MeshInstance3D y le di esta forma:



Cuando entremos en esta área nos mandará directamente a la área “localización”. El script asignado a “tp” es el siguiente”:

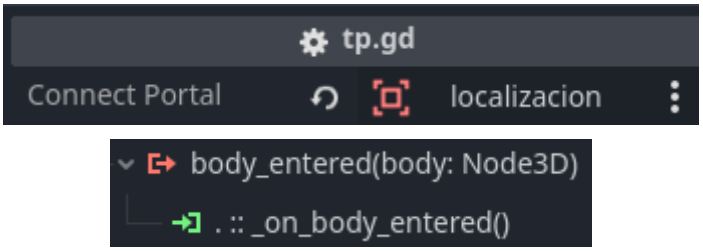
```gdscript
extends Area3D

@export var connect_portal: Area3D

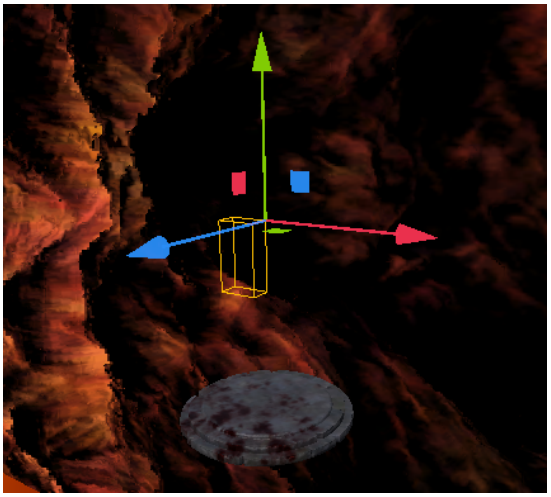
func _on_body_entered(body: Node3D) -> void:
    if body.name == "Jugador":
        var destination = connect_portal.global_transform.origin
        body.global_transform.origin = destination

```

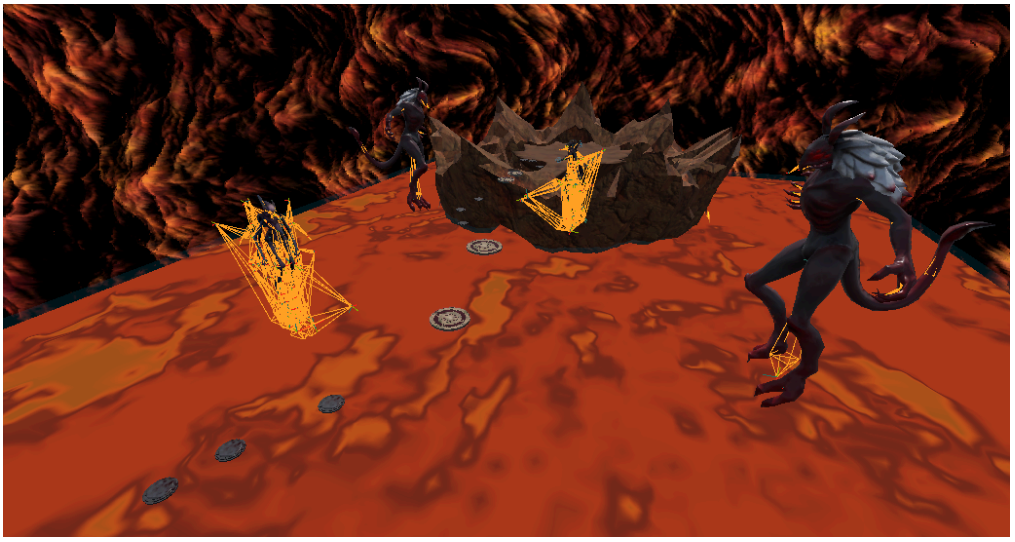
Conectamos un Area3D, que será la llamada “localización” que se posiciona en el otro mapa.

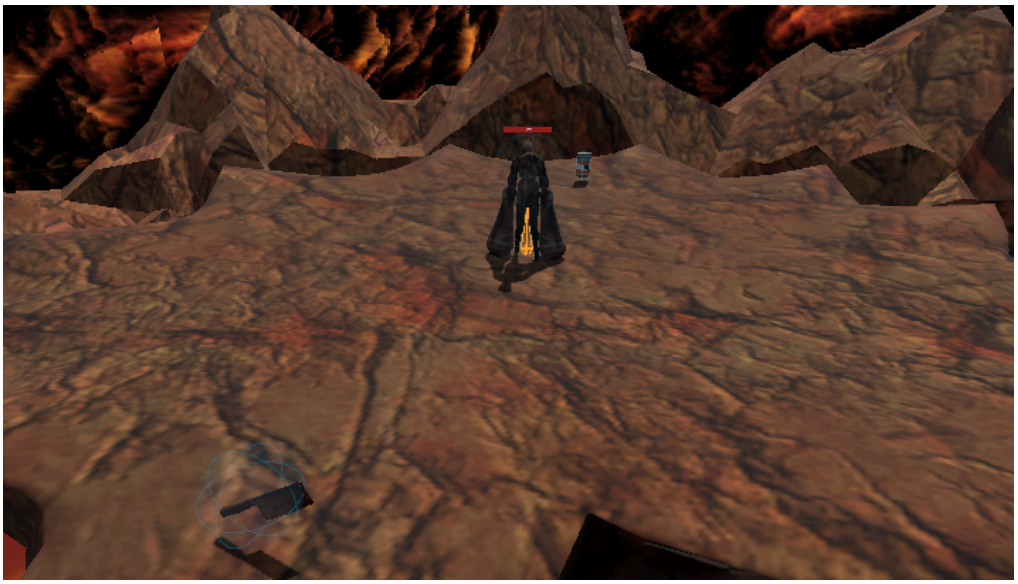


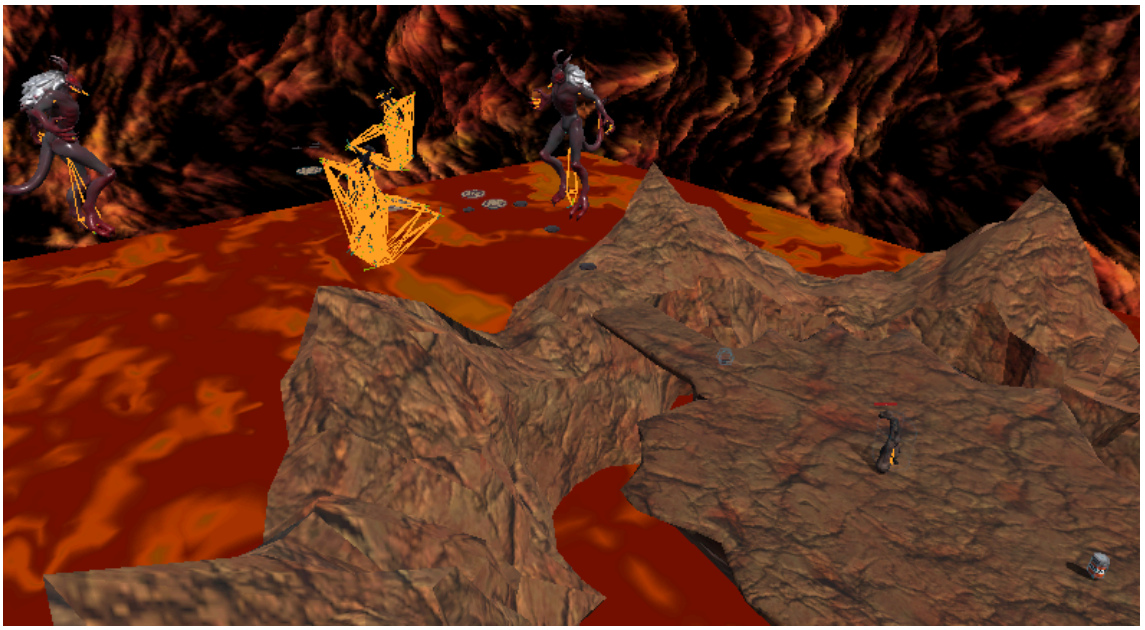
Y cuando entremos en “tp” siendo el jugador, transportamos al jugador a la posición de destino:



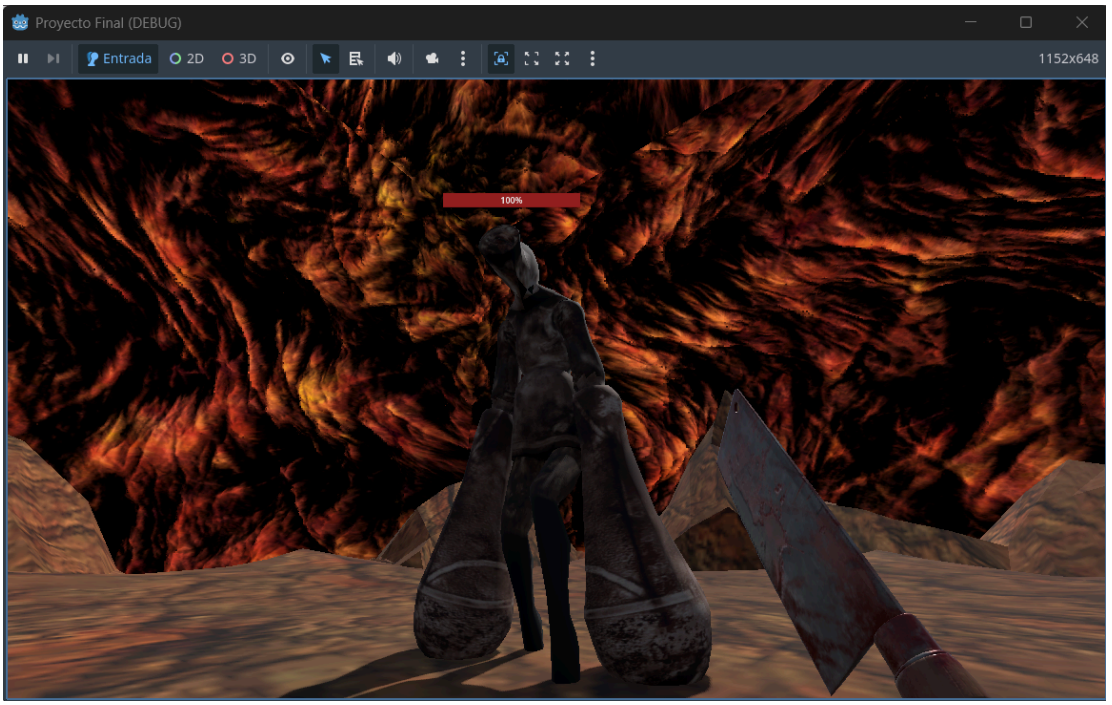
4. Mapa feo

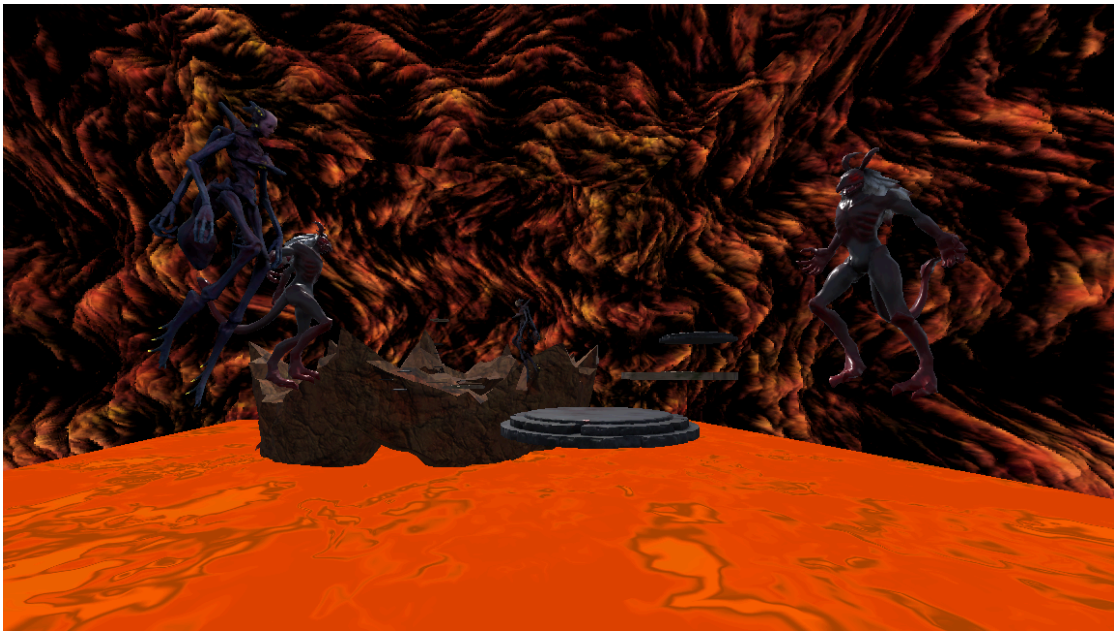




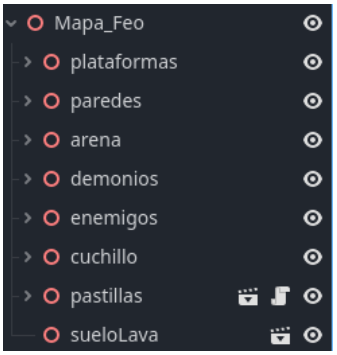


En partida:

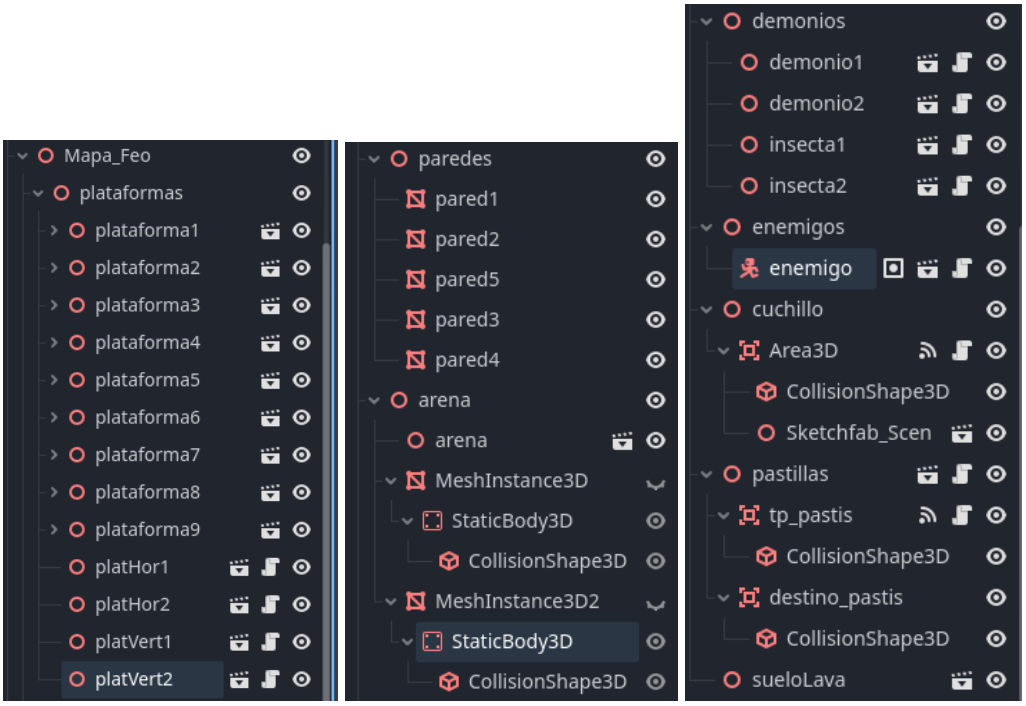




Estructura general del mapa:



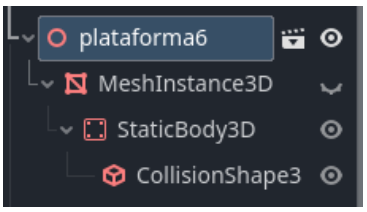
Estructura de cada nodo:

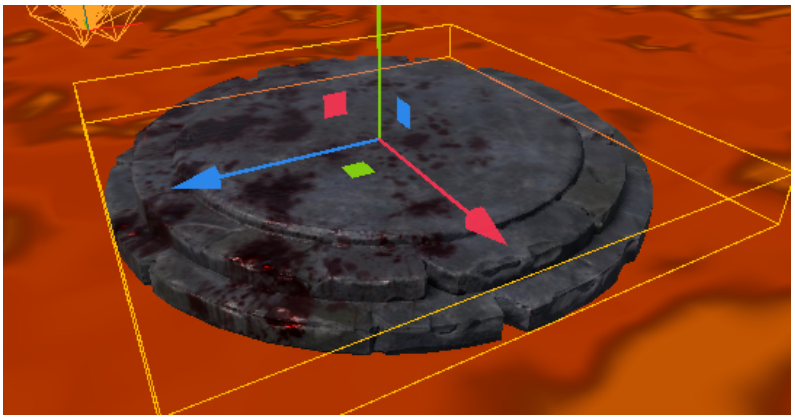


4.1 Plataformas

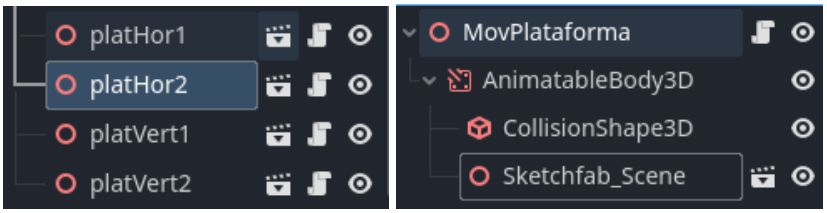
Tenemos 2 tipos de plataformas: estáticas y en movimiento.

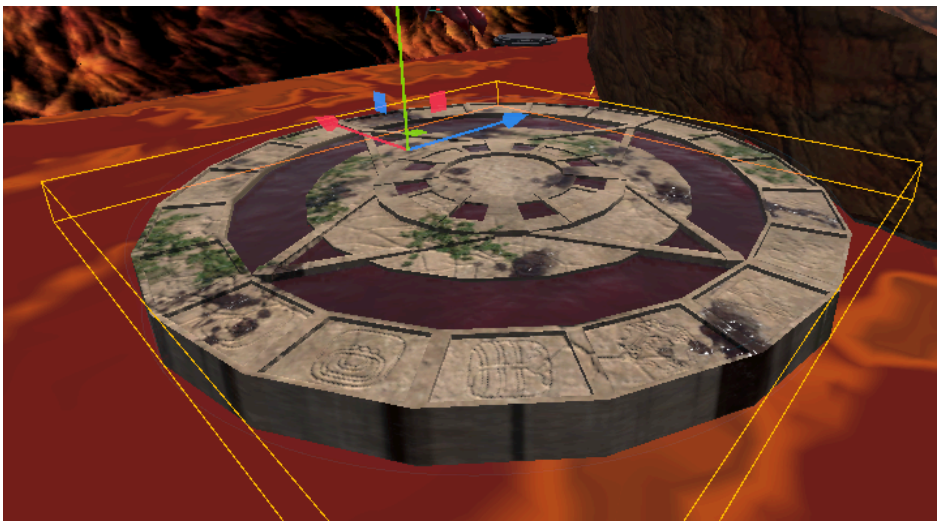
Las plataformas estáticas son simplemente modelos importados con un MeshInstance3D y un CollisionShape3D para poder pisarlos.





En las plataformas dinámicas tenemos dos tipos, las que se mueven horizontalmente, y las que se mueven verticalmente.





- El script de la plataforma horizontal:

```
1   extends Node3D
2
3   @export var distancia := 20.0
4   @export var velocidad := 0.5
5
6   var origen: Vector3
7   var t := 0.0
8
9   func _ready():
10      origen = $AnimatableBody3D.global_position
11
12  func _physics_process(delta):
13      t += delta * velocidad
14      var offset = -sin(t) * distancia
15      $AnimatableBody3D.global_position = origen + Vector3(0, 0, offset)
```

Establecemos una distancia de recorrido y una velocidad, la variable origen es la posición inicial de la plataforma y t el tiempo acumulado.
En la función _ready() guardamos la posición inicial, y en la función _physics_process calculamos el tiempo de avance, el offset que nos marca la distancia y movemos el objeto en el eje z.

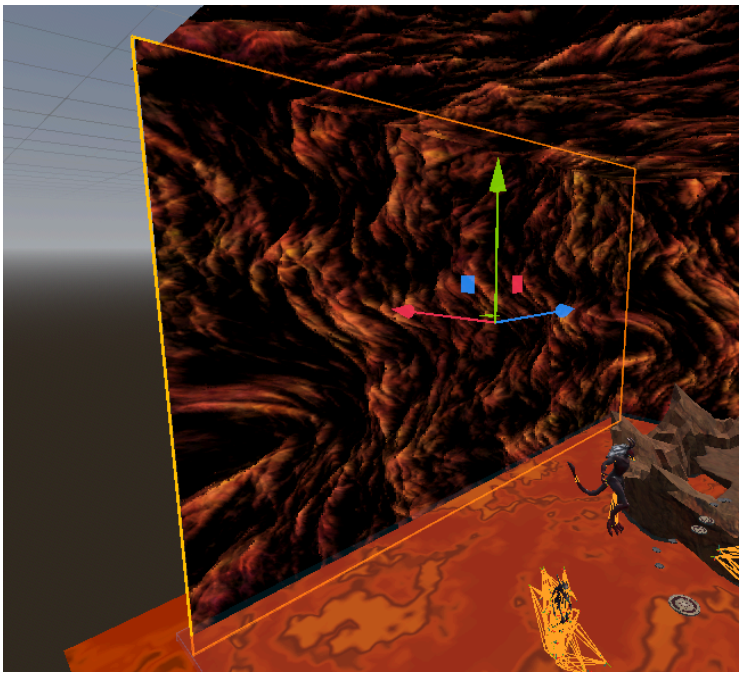
- El script de la plataforma vertical:

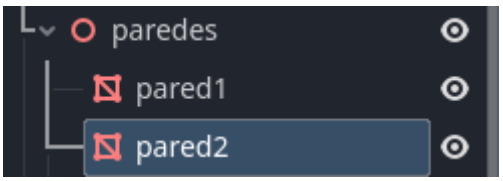
```
1   extends Node3D
2
3   @export var distancia := 10.0
4   @export var velocidad := 0.5
5
6   var origen: Vector3
7   var t := 0.0
8
9   func _ready():
10      origen = $AnimatableBody3D.global_position
11
12  func _physics_process(delta):
13      t += delta * velocidad
14      var offset = -sin(t) * distancia
15      $AnimatableBody3D.global_position = origen + Vector3(0, offset, 0)
16
```

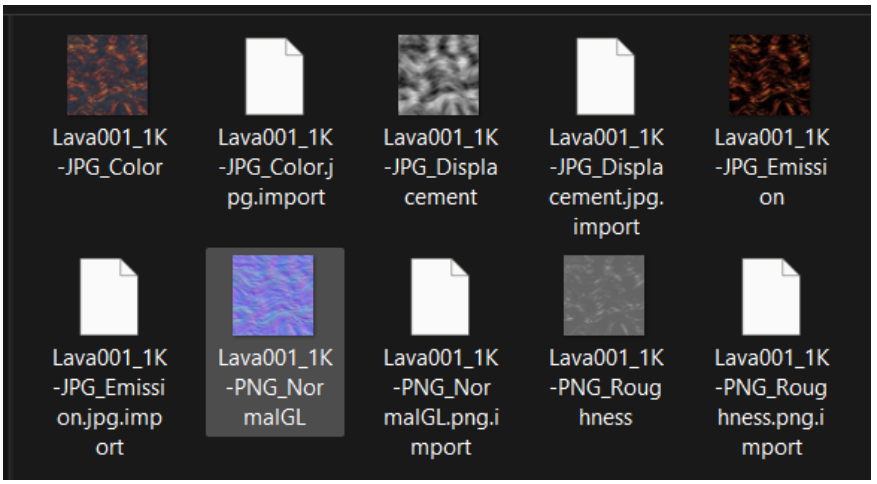
Igual que el horizontal pero el offset lo situamos en el eje y para que sea un movimiento vertical.

4.2 Paredes

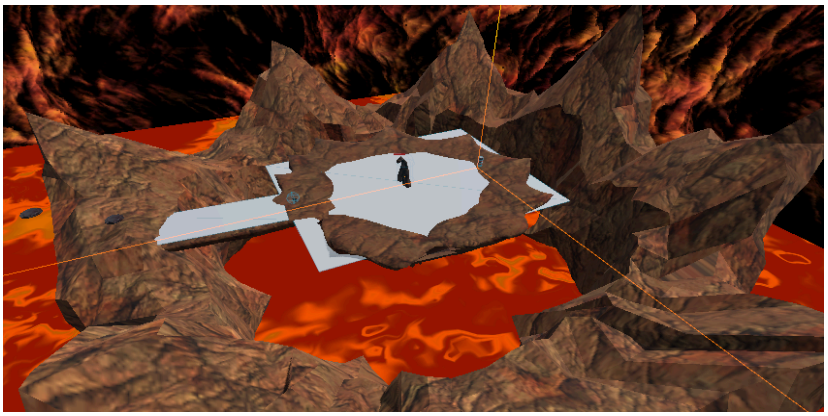
Las paredes las hago con un MeshInstance3D y le doy forma de un PlaneMesh, después le añado una textura para dar efecto de magma:

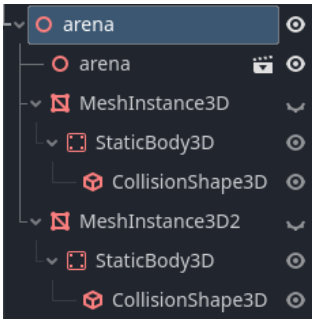






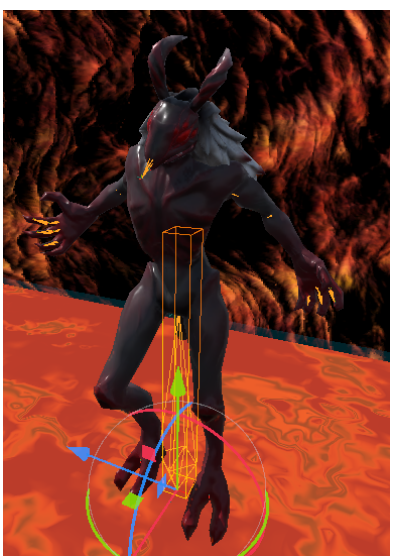
4.3 Arena

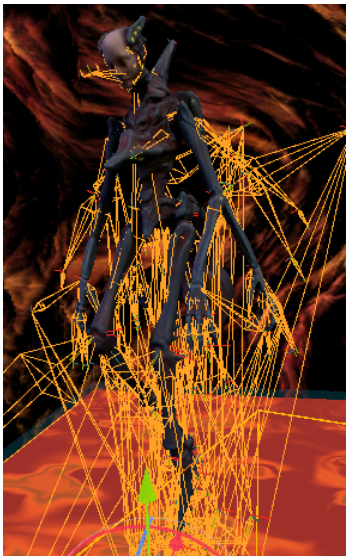


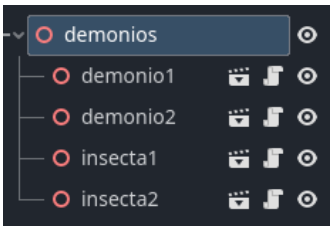


La arena es simplemente un modelo importado con dos MeshInstance3D con sus respectivos CollisionShape3D para poder ser pisados.

4.4 Demonios







Son cuatro modelos importados animados con sus respectivos scripts para permitir el movimiento:

```gdscript
extends Node3D

@onready var anim = $"AnimationPlayer"

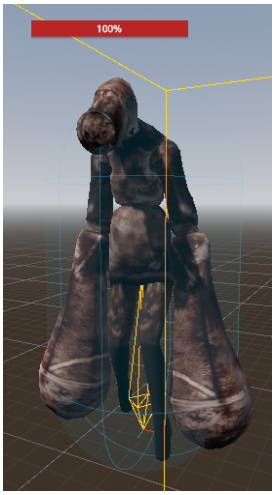
func _ready():
	anim.get_animation("Take 001").loop = true
	anim.play("Take 001")
```

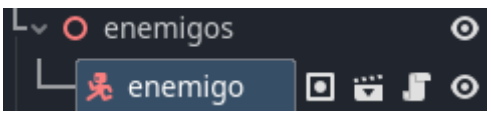
```gdscript
extends Node3D

@onready var anim = $"AnimationPlayer"

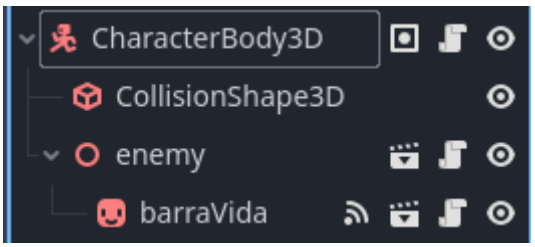
func _ready():
	anim.get_animation("rig|rigAction").loop = true
	anim.play("rig|rigAction")
```

4.5. El enemigo





Empezaremos con la escena del enemigo, que se ve así:



Tenemos un CharacterBody3D, este pertenece al grupo “enemy” y tiene un script asociado, además tiene un CollisionShape3D para hacer las colisiones con el rayo de nuestro jugador, también tenemos un modelo importado para el personaje, que también es animado y contiene el siguiente script asociado:

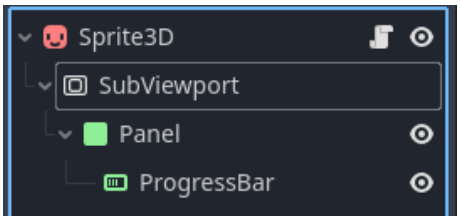
```
extends Node3D

@onready var anim = $"AnimationPlayer"

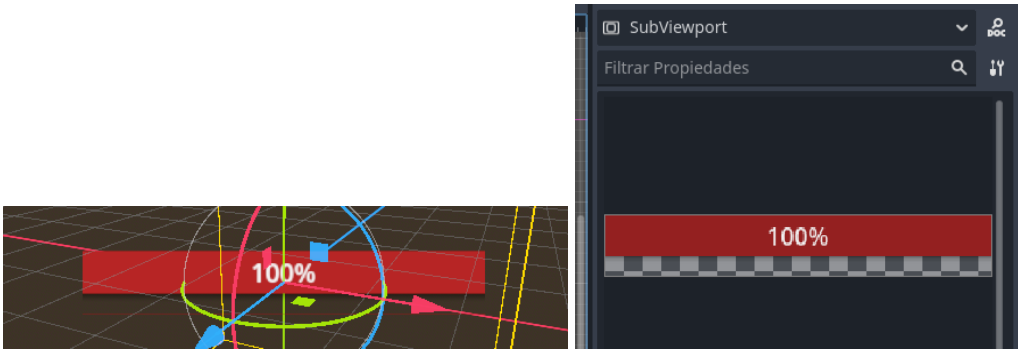
func _ready():
    anim.get_animation("Take 001").loop = true
    anim.play("Take 001")
```

Y además hay una barra de vida que nos servirá para indicar la vida del enemigo y poder hacerle daño.

Analicemos la escena de barraVida:



El Sprite3D nos permitirá que la forma plana de la barra de vida se vea en la escena 3D, SubViewport nos renderiza la escena 2D y el Panel junto con ProgressBar nos permite crear la barra de vida:



Aquí configuramos algunas características como el tamaño, los colores, la forma,

etc. Luego tenemos el siguiente script para el Sprite3D:

```gdscript
extends CharacterBody3D

@export var max_hp := 100.0
@export var hp := max_hp
var can_take_damage := true
var timeout := 1.0

@onready var barra: Sprite3D = $enemy/barraVida
@onready var pastillas: Node3D = $"../../pastillas"

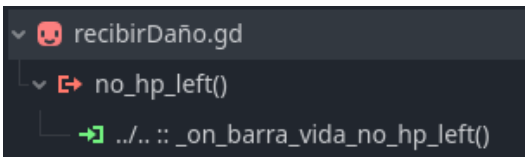
func _ready() -> void:
	hp = max_hp

func take_damage(amount: float) -> void:
	if not can_take_damage:
		return

	hp -= amount
	barra.take_damage(amount)
```

Tenemos una señal para cuando no quede hp (vida), un valor que indica el máximo de hp, y la barra de vida.

- En la función _ready(): Establecemos el valor máximo de la barra y el valor al inicio de la partida.
- En la función take_damage(...) le quitamos el valor damage a la barra y si el valor es menos o igual a 0.1, emitimos que no queda hp.



Ahora analicemos el script del propio enemigo:

```gdscript
extends CharacterBody3D

@export var max_hp := 100.0
@export var hp := max_hp
var can_take_damage := true
var timeout := 1.0

@onready var barra: Sprite3D = $enemy/barraVida
@onready var pastillas: Node3D = $enemy/barraVida
```

Una variable que nos indica el máximo hp, el hp, si puede recibir daño, un breve timeout, la barra de vida y las pastillas que aparecerán al morir.

- En la función _ready():

```
func _ready() -> void:
    hp = max_hp
```

- En la función take_damage(amount: float):

```
14 func take_damage(amount: float) -> void:
15     if not can_take_damage:
16         return
17
18     hp -= amount
19     barra.take_damage(amount)
20
21     if hp <= 0:
22         pastillas.show()
```

Si no puede recibir daño, no pasa nada, si puede se le resta el valor y se llama a take_damage de la barra con ese mismo valor, si se queda sin hp, mostramos las pastillas.

- En la función _on_barra_vida_no_hp_left():

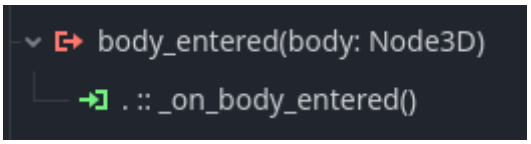
```
24 func _on_barra_vida_no_hp_left() -> void:
25     await get_tree().create_timer(timeout).timeout
26     queue_free()
```

Esperamos un momento, para que dé tiempo a que aparezcan las pastillas antes de que se elimine al enemigo, y lo eliminamos después.

4.6 El cuchillo



El cuchillo se constituye por un Area3D, que contiene un CollisionShape3D y un modelo importado de un cuchillo, cuando el jugador entra en el Area3D, el jugador obtiene el cuchillo.



El script del Area3D es el siguiente:

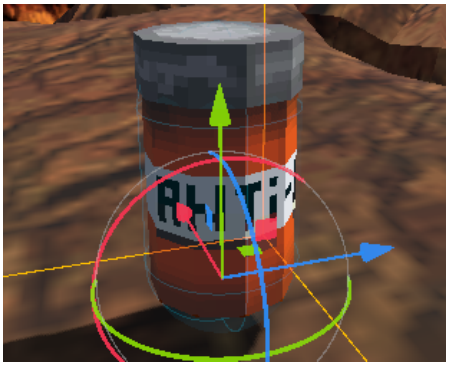
```gdscript
extends Area3D

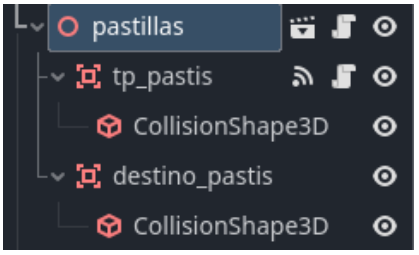
func _ready() -> void:
    add_to_group("pickup")

func _on_body_entered(body: Node3D) -> void:
    if body.is_in_group("player"):
        body.pickup_knife()
        queue_free()
```

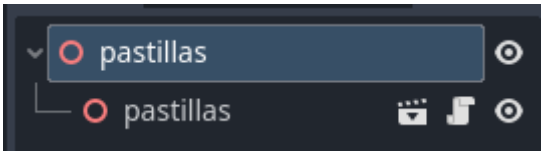
Añadimos al grupo pickup, y tenemos una señal que cuando alguien entre en el area, comprobamos si es el jugador y entonces llamamos a su función pickup_knife, y eliminamos.

4.7 Las pastillas





Las pastillas aparecen al morir el enemigo y al cogerlas nos teletransportamos al primer mapa.
La escena de las pastillas es la siguiente:



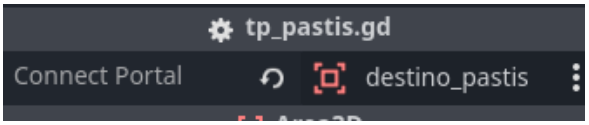
Que es un modelo importando con un script para el movimiento de este:

```
1   extends Node3D
2
3   @onready var anim = $"AnimationPlayer"
4
5   func _ready():
6       anim.get_animation("animation").loop = true
7       anim.play("animation")
```

Además tenemos asociadas 2 áreas, la primera es el área que rodea las pastillas, con el siguiente script:

```
1   extends Area3D
2
3   @export var connect_portal: Area3D
4   @onready var tp := $"../../../trasportaciones/tp"
5   func _on_body_entered(body: Node3D) -> void:
6       if body.name == "Jugador":
7           body.drop_knife()
8           tp.queue_free()
9           var destination = connect_portal.global_transform.origin
10          body.global_transform.origin = destination
```

Donde nos conectamos a las segunda área:



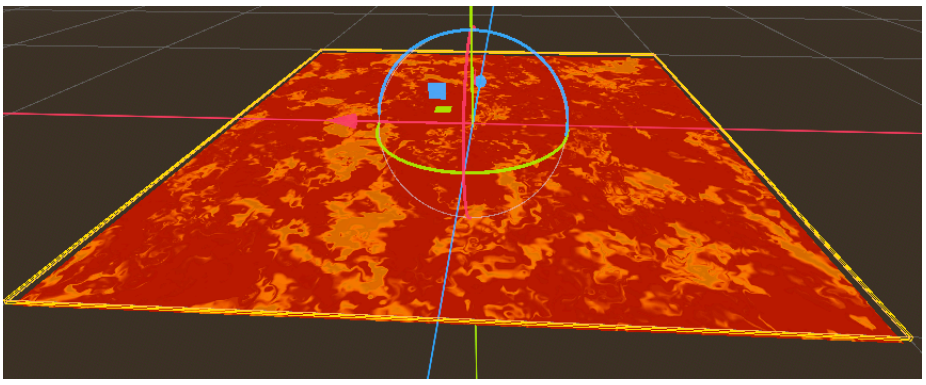
Y al “tp” del primer mapa, entonces cuando el jugador entre a esta área, nos quitaremos el cuchillo y eliminaremos “tp” para no poder volver y finalizar el juego, y entonces iremos a la localización de destino_pastis que es en el primer mapa.

Además el nodo pastillas tiene el siguiente script:

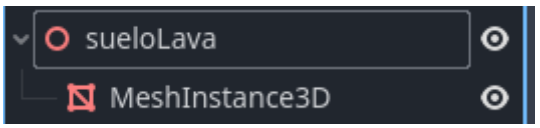
```
1   extends Node3D
2
3   func _ready() -> void:
4       self.visible = false
```

Para que al principio no sean visibles.

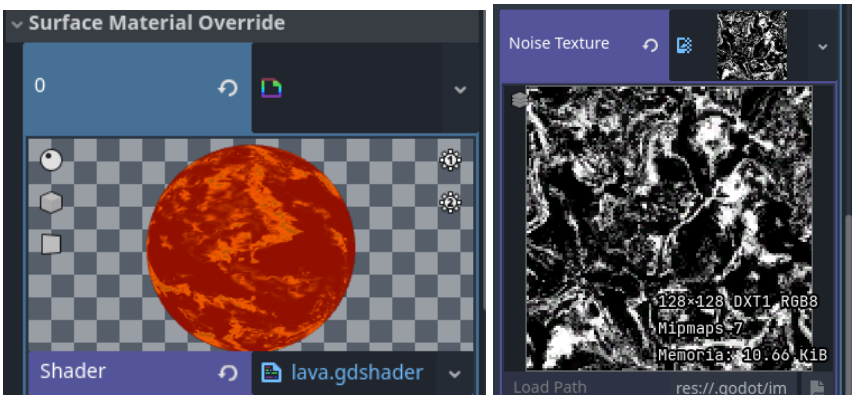
4.8 Suelo de lava



Para la lava del segundo mapa hice un shader.



Me ayudé de internet para crear el código y modifiqué algunas variables.

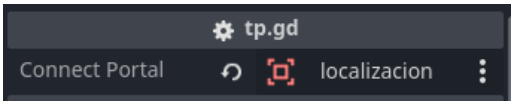


5. Teletransportación (2ª parte)

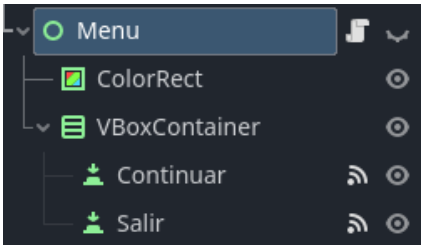


Nos queda hablar del área de lava, que cubre la parte donde se encuentra el suelo de lava del segundo mapa, así cuando el jugador entre en contacto con esta por caída volverá al inicio del mapa, osea al área localización.

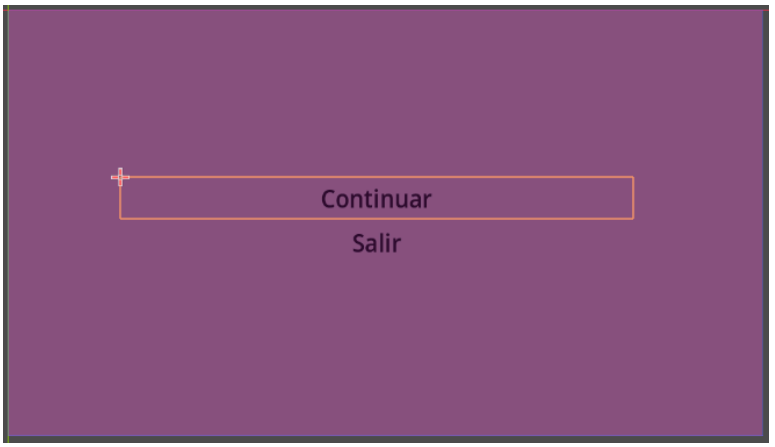
```
1  extends Area3D
2
3  @export var connect_portal: Area3D
4
5  func _on_body_entered(body: Node3D) -> void:
6      if body.name == "Jugador":
7          var destination = connect_portal.global_transform.origin
8          body.global_transform.origin = destination
```



6. Menú (Interfaz)



Menu es un nodo Control que es la base de UI, ColorRect es el fondo del menú y VBoxContainer coloca los botones uno debajo del otro automáticamente, finalmente tenemos 2 botones Continuar / Salir que reaccionan al ser clicados.



El script del menú es el siguiente:

```gdscript
extends Control

func _ready() -> void:
	visible = false

func mostrar():
	visible = true
	get_tree().paused = true

func ocultar():
	visible = false
	get_tree().paused = false

func _on_continuar_pressed() -> void:
	ocultar()
	Input.mouse_mode = Input.MOUSE_MODE_CAPTURED

func _on_salir_pressed() -> void:
	get_tree().quit()
```

- Función _ready()

El menú no es visible al principio.

- Función mostrar()
  Hace visible el menú y pausa el juego

- Función ocultar()
  Hace invisible el menú y despausa el juego

- Función _on_continuar_pressed()
  Llama a ocultar() y captura el ratón

- Función _on_salir_pressed()
  Cierra el juego

Aquí finalizamos la descripción del proyecto completa.